

Clean Code - FULL Class Design + DIP Guide

"Clean code is not written by following a set of rules. It is written by programmers who care."

Single Responsibility Principle (SRP)

Bad Example (God Class)

```
class UserManager:
    def login(self, username, password):
        print("Login logic")

    def register(self, username, password):
        print("Register logic")

    def send_email(self, email, message):
        print("Send email")

    def generate_report(self):
        print("Generate report")
```

Good Example (Separated Responsibilities)

```
class AuthService:
    def login(self, username, password):
        print("Login logic")

    def register(self, username, password):
        print("Register logic")

class EmailService:
    def send_email(self, email, message):
        print("Send email")

class ReportService:
    def generate_report(self):
        print("Generate report")
```

Cohesion

Bad Cohesion

```
class RandomStuff:
    def calculate_salary(self, hours):
        return hours * 10

    def send_email(self, msg):
        print(msg)

    def draw_ui(self):
        print("Draw UI")
```

Good Cohesion

```
class SalaryCalculator:
    def calculate_salary(self, hours):
        return hours * 10
```

Encapsulation

Bad

```
class User:
    name = "Arthur"
```

Good

```
class User:
    def __init__(self, name):
        self.__name = name

    def get_name(self):
        return self.__name
```

Classes Should Be Small

Bad

```
class SuperManager:
    def process_user(self): pass
    def process_order(self): pass
    def process_payment(self): pass
    def process_email(self): pass
    def process_report(self): pass
```

Good

```
class UserManager:
    def process_user(self): pass

class OrderManager:
    def process_order(self): pass
```

Open Closed Principle (OCP)

Bad

```
class Payment:
    def pay(self, method):
        if method == "paypal":
```

```

        print("Pay with PayPal")
    elif method == "card":
        print("Pay with Card")

```

Good

```

class Payment:
    def pay(self, amount):
        raise NotImplementedError

class Paypal(Payment):
    def pay(self, amount):
        print(f"Pay {amount} with PayPal")

class Card(Payment):
    def pay(self, amount):
        print(f"Pay {amount} with Card")

```

Dependency Inversion Principle (DIP)

Bad

```

class MySQL:
    def save(self, data):
        print("Saving to MySQL")

class App:
    def __init__(self):
        self.db = MySQL()

    def process(self):
        self.db.save("data")

```

Good (Full Implementation)

```

# Abstraction
class Database:
    def save(self, data):
        raise NotImplementedError

# Implementations
class MySQL(Database):
    def save(self, data):
        print("Saving to MySQL")

class PostgreSQL(Database):
    def save(self, data):
        print("Saving to PostgreSQL")

# High-level module
class App:
    def __init__(self, db: Database):
        self.db = db

    def process(self):
        self.db.save("data")

# Usage
db = MySQL()

```

```
app = App(db)
app.process()

# Switch database easily
db = PostgreSQL()
app = App(db)
app.process()

# Testing with fake dependency
class FakeDB(Database):
    def save(self, data):
        print("Fake save for testing")

test_app = App(FakeDB())
test_app.process()
```

Final Quote

"Programs must be written for people to read, and only incidentally for machines to execute." – Harold Abelson